學號： 614530036
姓名： 邱世宇 Email：jasonchiu0916@gmail.com

實驗報告：LAB8：Linux Device Driver I

一、實驗名稱

LAB8：Linux Device Driver I — Character Device Driver 實作

二、實驗目的

本實驗目的是了解 Linux Driver 的基本概念與運作方式，並實作一個可以被 kernel 掛載與卸載的基本 driver。

透過本實驗可學會以下重點：

- Kernel Module 的基本架構。
- 使用 module_init() 與 module_exit() 指定模組載入與卸載時執行的函式。
- 使用 insmod、rmmod、lsmod 測試 module 是否成功載入與卸載。
- 建立基本 Character Device Driver。
- 使用 register_chrdev() 註冊字元裝置。
- 建立 /dev/demo，讓 user program 可以透過檔案介面存取 driver。
- 使用 dmesg 觀察 driver 在 kernel space 中輸出的 printk 訊息。

三、實驗步驟

1. 建立簡單 Kernel Module

撰寫 hello.c，內容包含 demo_init() 與 demo_exit()。demo_init() 會在 module 被載入時執行，並用 printk() 輸出訊息；demo_exit() 會在 module 被卸載時執行，並用 printk() 輸出訊息。

2. 撰寫 Makefile

建立 Makefile，指定要編譯的 module。若檔案名稱為 hello.c，則使用 obj-m := hello.o。

```
obj-m := hello.o

all:
	make -C /home/jasonchiu/WORK/buildroot/output/build/linux-custom M=$(PWD) modules

clean:
	make -C /home/jasonchiu/WORK/buildroot/output/build/linux-custom M=$(PWD) clean
```

3. 編譯 Kernel Module

在 module 目錄下使用 ARM64 cross compiler 編譯，完成後會產生 hello.ko。

```
make ARCH=arm64 CROSS_COMPILE=/home/jasonchiu/WORK/buildroot/output/host/bin/aarch64-linux-
```

學號： 614530036
姓名： 邱世宇 Email：jasonchiu0916@gmail.com
產生檔案：hello.ko

4. 複製 Module 到 Raspberry Pi

將 hello.ko 複製到 Raspberry Pi 的 root filesystem，例如 /root 或其他可執行測試的位置。

5. 載入與卸載 Module

在 Raspberry Pi 上使用 insmod 載入 module，使用 lsmod 確認 module 是否存在，使用 rmmod 卸載 module，最後用 dmesg 查看 kernel log。

```
insmod hello.ko
lsmod
dmesg
rmmod hello
dmesg
```

6. 擴充為 Character Device Driver

在 hello.c 中加入 drv_open()、drv_release()、drv_read()、drv_write()、drv_ioctl() 等 driver function。

```
drv_open()
drv_release()
drv_read()
drv_write()
drv_ioctl()
```

7. 建立 file_operations 結構

建立 struct file_operations drv_fops，將 user program 的 open、read、write、ioctl、close 操作對應到 driver function。

8. 註冊 Character Device

在 demo_init() 中使用 register_chrdev() 註冊字元裝置；在 demo_exit() 中使用 unregister_chrdev() 解除註冊。

```
#define MAJOR_NUM 60
register_chrdev(MAJOR_NUM, "demo", &drv_fops);
unregister_chrdev(MAJOR_NUM, "demo");
```

9. 建立 Device Node

在 Raspberry Pi 上建立 /dev/demo，使 user program 可以透過檔案路徑存取 driver。

```
mknod /dev/demo c 60 0
chmod 666 /dev/demo
```

10. 撰寫測試程式 test.c

撰寫 user program，透過 open()、read()、write()、close() 操作 /dev/demo，測試 driver function 是否被觸發。

```
open("/dev/demo", ...);
read(...);
```

學號： 614530036
姓名： 邱世宇 Email：jasonchiu0916@gmail.com

```
write(...);
close(...);
```

11. 編譯並執行 test.c

使用 ARM64 cross compiler 將 test.c 編譯成 test.exe，複製到 Raspberry Pi 後執行。

```
/home/jasonchiu/WORK/buildroot/output/host/bin/aarch64-linux-gcc -static -g test.c -o test.exe
```

12. 觀察 dmesg 結果

執行 test.exe 後使用 dmesg 確認 kernel log 中是否出現 device open、device read、device write、device close 等訊息。

```
device open
device read
device write
device close
```

四、Q&A：問題與討論

Q1：MODULE_LICENSE() 是做什麼用的？

MODULE_LICENSE() 用來宣告 kernel module 的授權條款，例如 GPL。它可以讓 Linux kernel 知道此 module 的授權狀態。如果沒有設定 GPL 授權，kernel 可能會將 module 標記為 proprietary module，且某些 GPL-only 的 kernel symbol 可能無法使用。

Q2：MODULE_DESCRIPTION() 是做什麼用的？

MODULE_DESCRIPTION() 用來描述這個 kernel module 的功能，讓使用者或開發者可以快速知道此 module 的用途，方便維護與辨識。

Q3：MODULE_AUTHOR() 是做什麼用的？

MODULE_AUTHOR() 用來標示 kernel module 的作者資訊，讓其他開發者知道此 module 的撰寫者，方便日後維護、追蹤與除錯。